```
concierge-key")    ❶
  ...
}
```

❶ Key name that makes the key unique

When a VIP guest registers at the hotel, the person at the front desk enters the guest's data into the hotel app. This creates the actor representing the guest and sends the `EnterHotel` message to that actor, which triggers its registration to the conci- erge `goldenKey` by sending a `Register` message to the `context.system .receptionist`, as shown in the following snippet. Note that the `Register` message requires a `ServiceKey` and the reference to the actor to be registered:

```
import akka.actor.typed.scaladsl.Behaviors
import akka.actor.typed.receptionist.{ Receptionist, ServiceKey }

object VIPGuest {

  sealed trait Command
  final case object EnterHotel extends Command

  def apply() = Behaviors.receive[Command] { (context, message) =>
    message match {
      case EnterHotel =>
        context.system.receptionist ! Receptionist            ❶
          .Register(HotelConcierge.goldenKey, context.self)
        Behaviors.same
    }
}}
```

❶ Registers to HotelConcierge .goldenKey on entering

On the other hand, the concierge actor subscribes to the key instead of registering, but it needs an adapter because the notification it receives does not belong to its protocol. It belongs to the receptionist API and is an `akka.actor.typed.receptionist`